LAB-02: ARP Fundamentals

Table of Contents

1. Objective

This lab investigates how ARP (Address Resolution Protocol) works in practice, why it exists, what exactly it sends on the wire, what the ARP cache is and how it affects network behavior, and how ARP fits into every single network communication that happens at Layer 2. ARP is the invisible plumbing under every ping, every TCP connection, every DNS query understanding it is foundational to understanding all later labs.

2. Why ARP Exists

IP addresses are logical they're assigned by software and can change. MAC addresses are physically burned into hardware (though they can be spoofed). The problem: the Internet Protocol uses IP addresses to route packets globally, but Ethernet uses MAC addresses to deliver frames locally. When two devices are on the same subnet and one wants to send a packet to the other, it knows the destination IP (from the user's command or from DNS) but it does not know the destination MAC. It cannot build the Ethernet frame without it.

ARP solves this: "I know the IP I want to reach. What's the MAC address for that IP?"

Task 1: Clear and observe ARP fresh

Why ip neigh flush all first:

ip neigh refers to the ARP cache (the kernel's table of IP → MAC mappings learned from previous ARP exchanges). flush all deletes every entry. This is done to guarantee that Kali has no prior knowledge of Metasploitable2's MAC address, so it is forced to send an actual ARP request on the wire when you ping. Without flushing, Kali might already know Metasploitable2's MAC from a previous lab session and skip the ARP exchange entirely you'd miss what you're trying to observe.

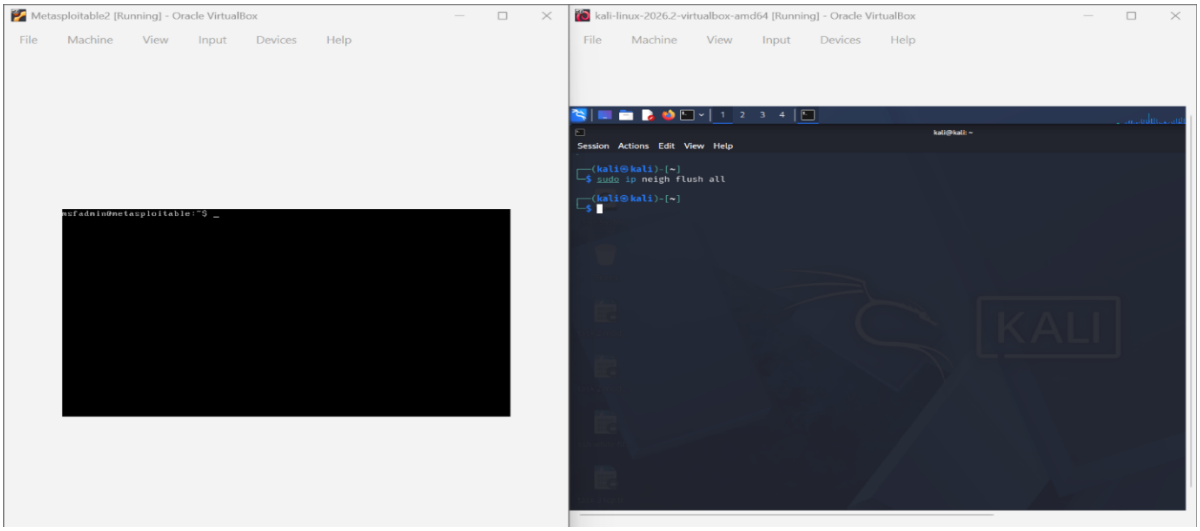
What happened in Wireshark (capture filter: arp):
Two ARP packets appear:

1. **ARP Request (broadcast):** Sent by Kali. Destination MAC: ff:ff:ff:ff:ff:ff (broadcast — every device on the segment receives this). The packet says in plain terms: "Who has 192.168.56.20? Tell 192.168.56.10." Kali knows its own IP and MAC, and it knows the target IP, but it does not know the target MAC so the Target MAC field in the request is 00:00:00:00:00:00 (empty/unknown).

2. **ARP Reply (unicast):** Sent by Metasploitable2 directly back to Kali. Destination MAC: Kali's MAC (unicast, only Kali receives this). The packet says: "192.168.56.20 is at 08:00:27:xx:xx:xx" Metasploitable2 answers with its own MAC address.

After this exchange, Kali knows Metasploitable2's MAC. It stores this in the ARP cache and proceeds to send the ICMP ping inside an Ethernet frame addressed directly to that MAC.

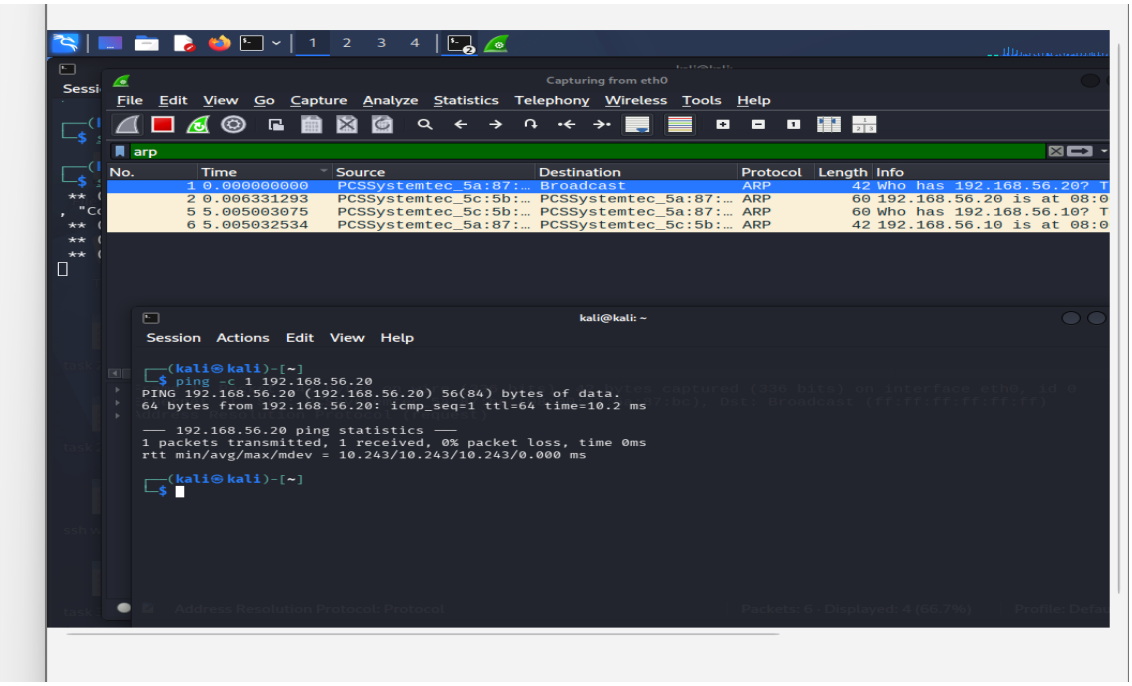
On Kali: sudo ip neigh flush all (clears ARP cache).

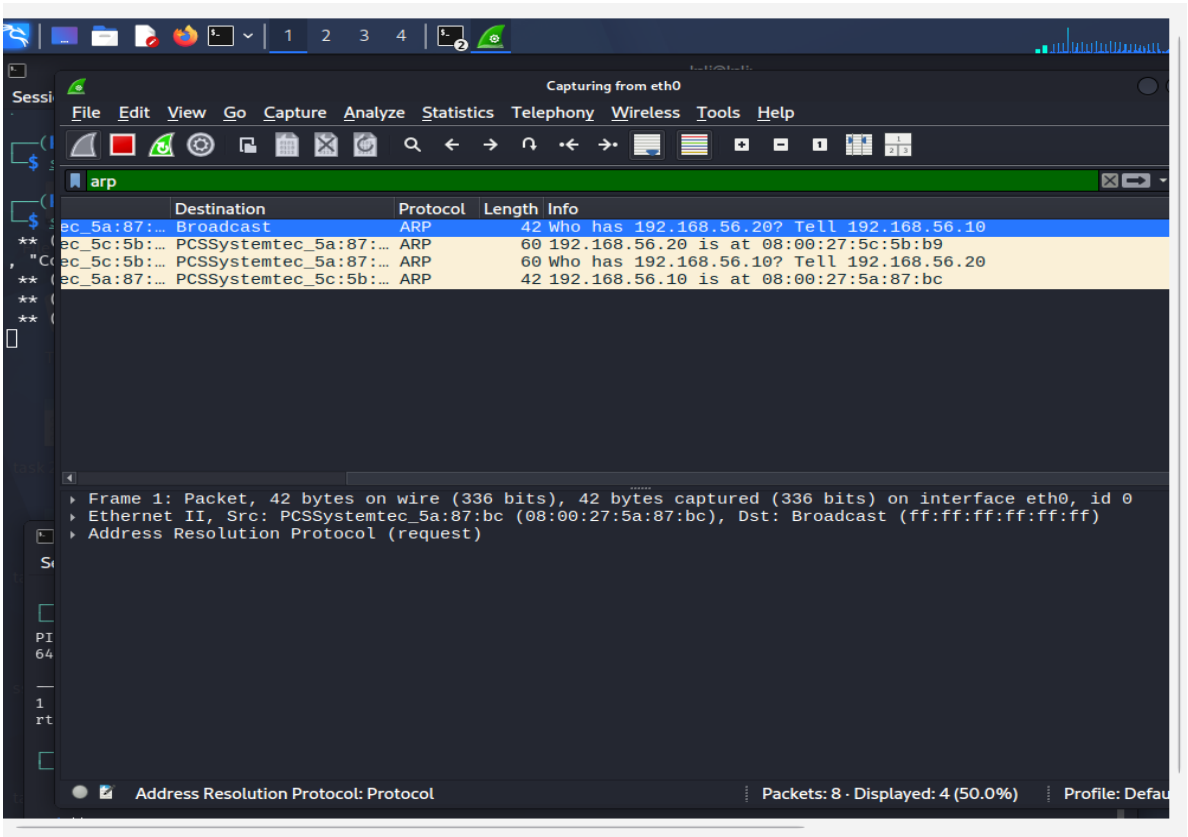


Start Wireshark capture with filter arp.

From Kali: ping -c 1 <metasploitable-ip>.

Stop capture. Screenshot the ARP Request (who-has) and ARP Reply (is-at) pair.





Task 2: Inspect ARP cache

On Kali: ip neigh (or arp -a) screenshot the cache entry now populated from task 1 last step

Command used:

bash

ip neigh

(Alternatively arp -a older syntax, same data)

What the output shows:

An entry like:

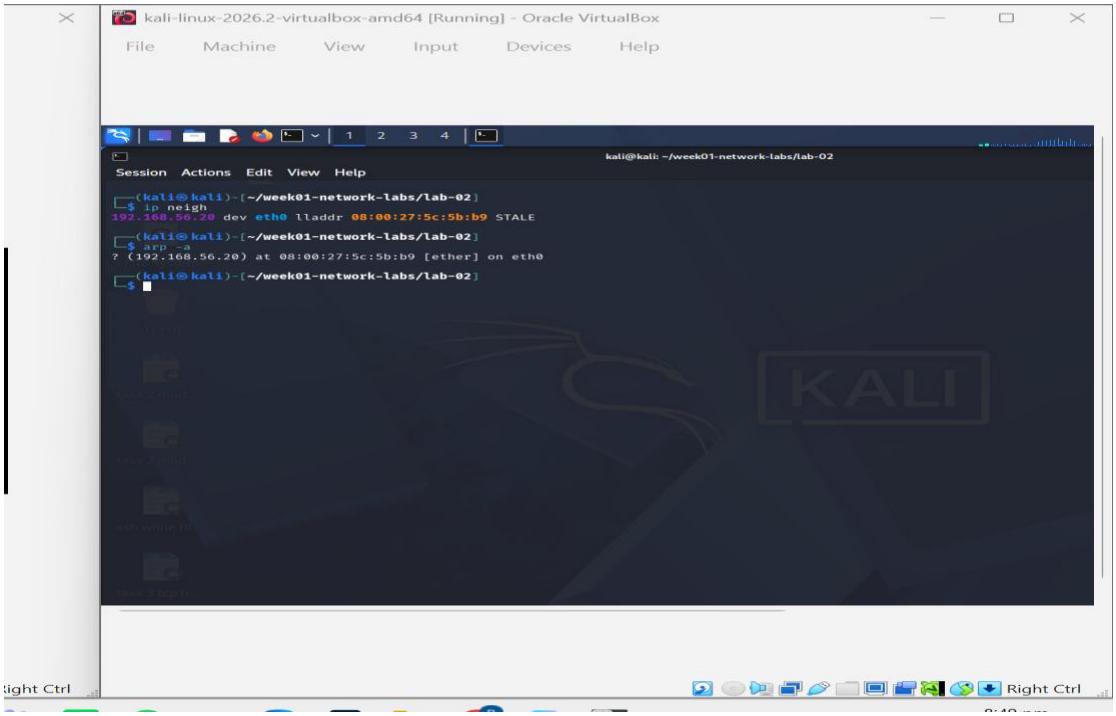
192.168.56.20 dev eth0 lladdr 08:00:27:xx:xx:xx REACHABLE

This reads as: "For IP 192.168.56.20, reachable via interface eth0, the MAC address (link-layer address) is 08:00:27:xx:xx:xx, and the entry is currently REACHABLE (recently confirmed active)."

Cache entry states:

- REACHABLE: the mapping was learned recently and is considered valid. Linux keeps ARP entries reachable for approximately 30 seconds after last confirmation.
- STALE: the entry exists but hasn't been confirmed recently. Will be re-verified the next time traffic is sent to that IP.
- FAILED: ARP request was sent but never answered (no such host, or host is down).

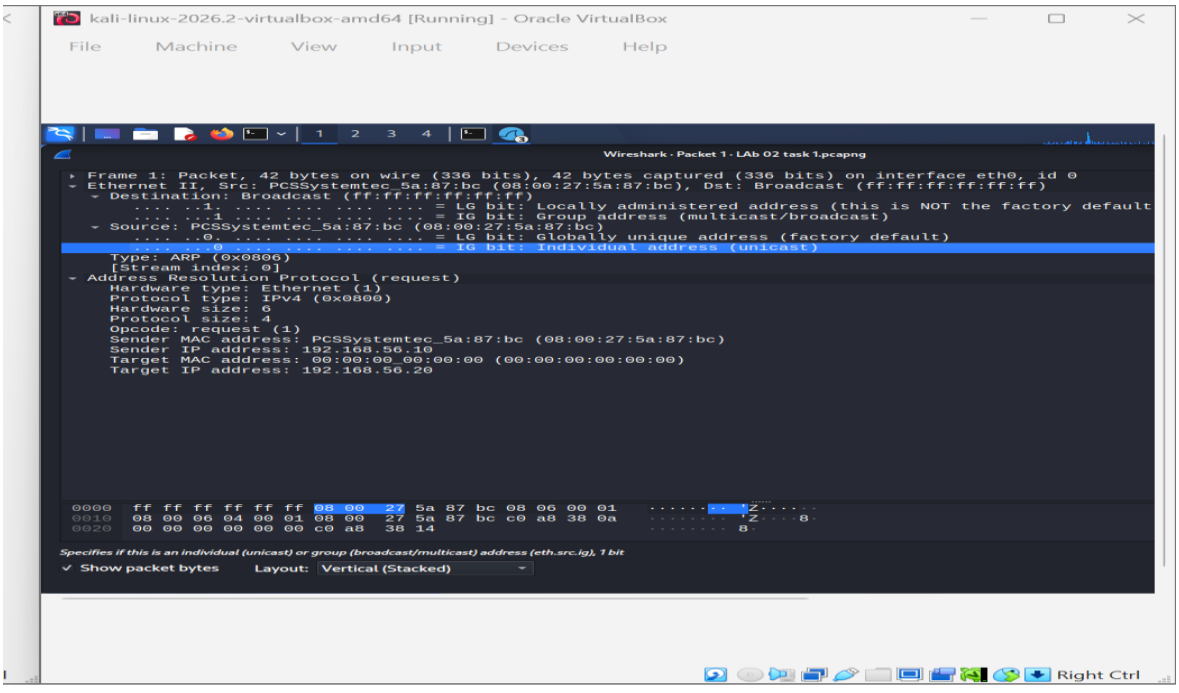
Why ARP caching matters: Without caching, every single packet sent to any IP on the local network would require an ARP exchange first. A web page load involves hundreds of packets — this would be catastrophically slow. The cache avoids this by remembering recent IP→MAC mappings for tens of seconds to minutes.



Task 3: ARP packet structure

Open the ARP Request packet from Task 1 in Wireshark, expand it fully.

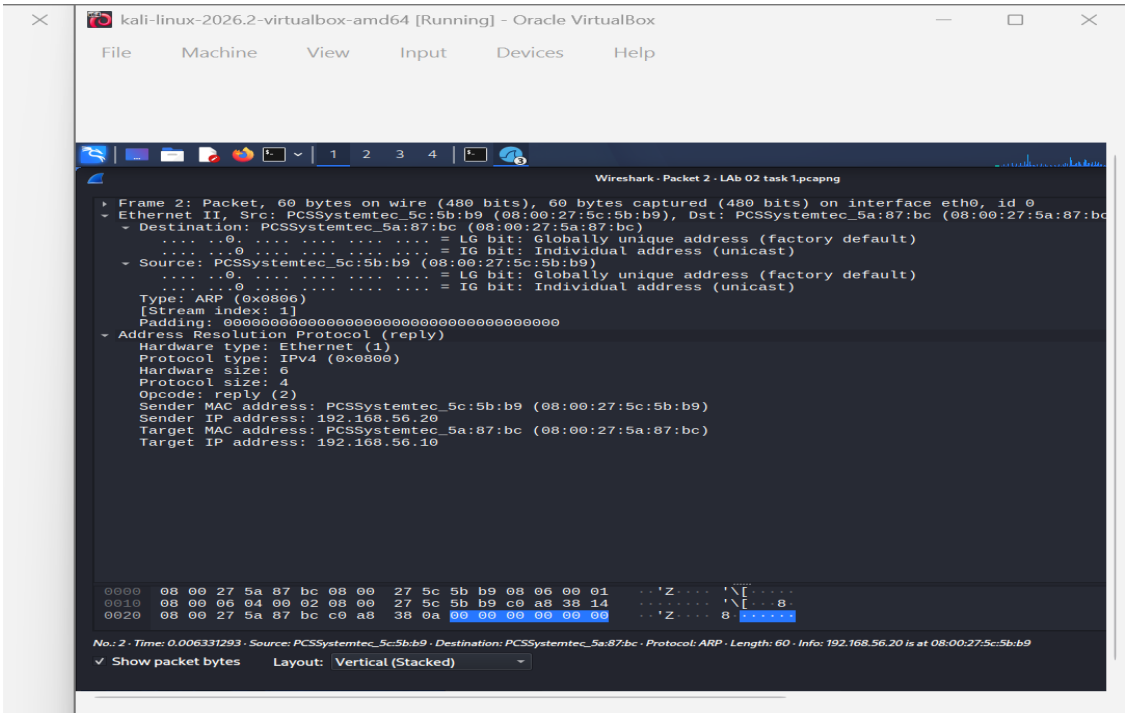
ARP request:



Field	Value	Meaning
Hardware Type	1 (Ethernet)	ARP is being used over an Ethernet network
Protocol Type	0x0800 (IPv4)	Resolving an IPv4 address
Hardware Size	6	MAC addresses are 6 bytes long
Protocol Size	4	IPv4 addresses are 4 bytes long
Opcode	1 (Request)	This is an ARP Request, not a Reply
Sender MAC	Kali's MAC	Kali identifies itself
Sender IP	192.168.56.10	Kali's IP
Target MAC	00:00:00:00:00:00	Unknown — this is what we're asking for
Target IP	192.168.56.20	The IP we want to resolve

Why Target MAC is 00:00:00:00:00:00 in the request: This is the whole point of the ARP Request Kali does not know the target MAC. The field is set to all zeros to indicate "unknown." The entire purpose of sending this broadcast is to get an answer to fill this field.

ARP reply:



Field	Value	Meaning
Hardware Type	1 (Ethernet)	Same
Protocol Type	0x0800 (IPv4)	Same
Hardware Size	6	Same
Protocol Size	4	Same
Opcode	2 (Reply)	This is an ARP Reply
Sender MAC	Metasploitable2's MAC	Metasploitable2 identifies itself with its actual MAC
Sender IP	192.168.56.20	Metasploitable2's IP
Target MAC	Kali's MAC	Reply goes back to Kali specifically
Target IP	192.168.56.10	Kali's IP

Task 4: Gratuitous ARP / broadcast behavior

Note the destination MAC address on the ARP Request frame (should be ff:ff:ff:ff:ff:ff) vs the ARP Reply's destination MAC (should be unicast, back to Kali specifically). Screenshot both, compare.

What was observed:

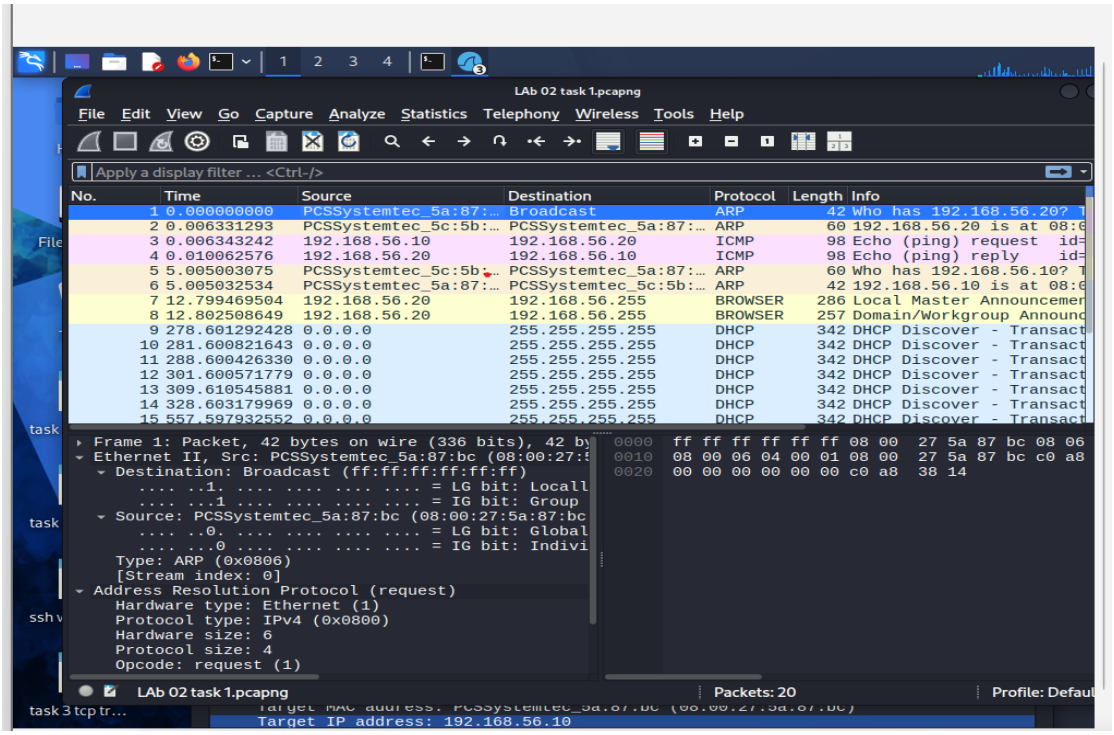
- ARP Request: Destination MAC in the Ethernet frame header = ff:ff:ff:ff:ff:ff (broadcast)
- ARP Reply: Destination MAC in the Ethernet frame header = Kali's MAC (unicast)

Why the request uses broadcast: Kali literally does not know which device on the network has the IP it's looking for it could be any of them. So it broadcasts to everyone simultaneously. Every device on the Internal Network segment receives and reads the ARP Request. Only the device that recognizes its own IP in the Target IP field responds.

Why the reply uses unicast: Metasploitable2 knows exactly who to reply to the Sender MAC from the request tells it. There is no reason to broadcast the reply to everyone. It sends it directly (unicast) to Kali's MAC.

Security note (for future labs): ARP has no authentication whatsoever. Any device can send an ARP Reply claiming to be any IP address. This is the basis of ARP Spoofing / ARP Poisoning

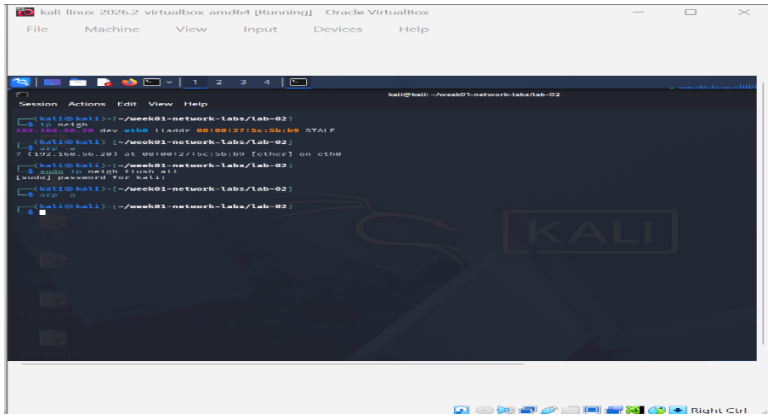
attacks an attacker sends fake ARP Replies to poison other devices' caches, redirecting traffic through the attacker's machine. This will be demonstrated in a later lab.

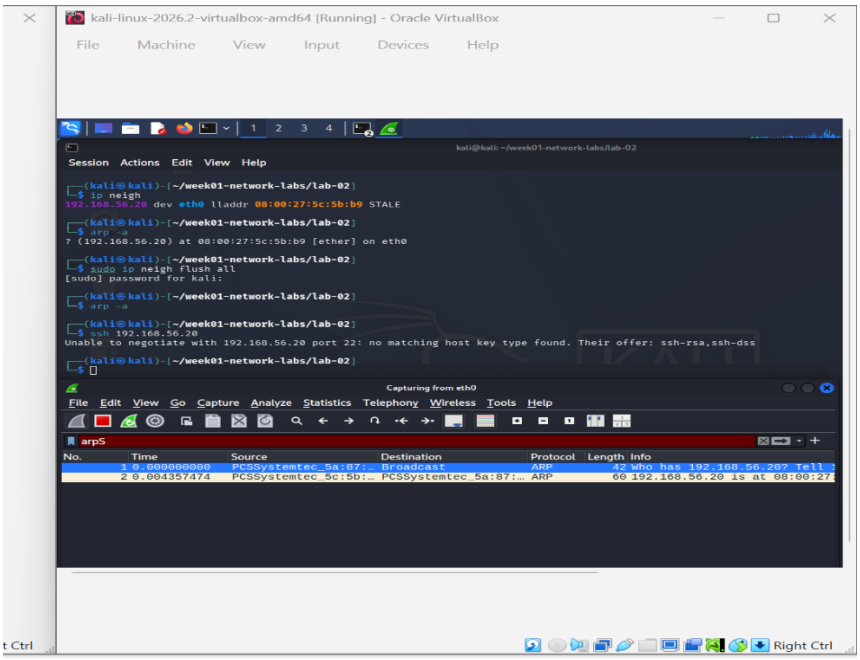


Task 5: ARP without prior contact

sudo ip neigh flush all again on Kali.

This time, instead of pinging, just try ssh <metasploitable-ip> (or any TCP-based command) and capture confirm ARP resolution happens automatically before the TCP/SSH traffic, even though you didn't ping first. Screenshot the ARP exchange followed by the TCP SYN.





What was done:

sudo ip neigh flush all # clear cache again

ssh 192.168.56.20 # try SSH to Metasploitable2

Wireshark was capturing simultaneously.

What was observed: Before any TCP/SSH traffic appears, an ARP Request is sent by Kali for Metasploitable2's MAC address. Only after the ARP Reply comes back and Kali knows the MAC does the TCP SYN (the first packet of the SSH handshake) appear in the capture.

Why this happens: SSH is a TCP application. TCP is a Layer 4 protocol. But to actually put a TCP SYN packet on the wire, Kali needs to build an Ethernet frame and that requires a destination MAC. Kali's networking stack automatically triggers ARP resolution for the destination IP the moment any application (ping, ssh, curl, nmap, anything) tries to send a packet to a local IP it doesn't have a MAC for. The application never knows this happened it's handled transparently by the kernel.

The key insight: ARP happens automatically, invisibly, before every Layer 3+ communication with a device on the local subnet not just before pings. When you run a nmap scan in lab-07, ARP will fire before every TCP probe. When you exploit Metasploitable2 in later labs, ARP will fire at the start of every session. It is the foundation of all local network communication.